AI Agentic Engineer — Take-Home Assignment

Designing an agentic system for ERP-driven financial statement generation

Time budget	6–8 hours of effort, spread over up to 5 calendar days
Submission	Architecture doc (PDF/MD) + Git repo + 1-page reflection
AI tools	Encouraged. Tell us how you used them.
Pre-start questions	You may email up to 3 clarifying questions before starting

1. Product context

We are building an AI-native financial reporting platform. One of its ten core features ingests a **trial balance** from an ERP (SAP, NetSuite), combines it with **manual adjustments** and a configurable **chart of accounts (COA)**, and produces four primary statements:

- **Balance Sheet** — assets, liabilities, equity at a point in time
- **Profit & Loss** — revenue, expenses, net income for the period
- **Cash Flow Statement** — operating, investing, and financing cash movements
- **Statement of Changes in Equity (SOCIE)** — opening to closing equity walk

The hard part is not the arithmetic. It is doing this reliably across messy real-world inputs: inconsistent COA mappings between ERPs, mid-period adjustments, intercompany eliminations, FX revaluation, prior-period restatements, and **audit-grade traceability** where every number on every statement reconciles back to source entries.

You will not build this product. We want to evaluate how you think about it.

2. The assignment

Design — don't fully build — an agentic system that takes a trial balance (CSV/JSON), a chart of accounts, and a list of manual adjustments, and produces the four financial statements. You may use any AI tools (Claude, Cursor, Copilot, etc.) and we expect you to. Tell us how you used them.

3. Inputs we provide

Alongside this PDF you will receive a small bundle of realistic, intentionally messy mock data. Do not assume it is clean — that is the point.

File	Contents	Known issues seeded
trial_balance.csv	Period-end TB from a fictional NetSuite-style export, ~80 accounts, multi-currency	Unbalanced after FX rounding; one duplicate account code; one orphan account not in COA
chart_of_accounts.csv	Hierarchical COA with statement mapping (BS/PL) and cash-flow category	Two accounts have ambiguous category; one node has no children mapped
manual_adjustments.json	10 journal entries from finance team (accruals, reclasses, FX reval, intercompany)	One entry has debit ≠ credit; one references an account not in COA; one is a circular IC entry

prior_period_tb.csv	Prior-period TB for opening balances and comparatives	Slightly different account list than current period (one account renamed)
fx_rates.csv	Period-average and period-end rates for non-functional currencies	One currency has missing period-end rate

We deliberately did not list every defect. Part of what we are evaluating is what you notice and how you handle it.

4. Deliverables

4.1 Architecture document — the main artifact we evaluate

Cover, at minimum:

- The **agent topology** you would use. Single agent with tools? Orchestrator with sub-agents (e.g., Mapper, Adjuster, Statement Builder, Validator)? Justify the choice.
- Where you draw the line between **deterministic code** and **LLM reasoning**. Generating a balance sheet from a clean TB is arithmetic — where does the AI actually earn its keep?
- How you handle the **failure modes** that will kill you in production: hallucinated account mappings, debits ≠ credits after adjustments, an account that fits no COA node, FX gaps, circular intercompany entries.
- Your **validation and self-correction loop**. What does the agent check before returning output? What does it do when checks fail?
- How an auditor traces any cell on the balance sheet back to source rows.

4.2 Working prototype of ONE slice

Pick the slice you find most interesting and build it end-to-end. **Do not attempt all four.** We would rather see one slice done thoughtfully than four done shallowly.

- **TB → COA mapper** that handles unmapped or ambiguous accounts with confidence scores and human-in-the-loop escalation.
- **Manual adjustments agent** that validates entries, detects balance violations, and explains rejections in plain English to a finance user.
- **Statement generator** for P&L + Balance Sheet with a verification agent that catches its own errors before output.
- **Reconciliation agent** that, given two TBs (e.g., pre-adjustment vs post-adjustment), explains every delta line by line.

Use the messy mock data we provided. Do not sanitize it first — handling the mess *is* the work.

4.3 One-page reflection

- What you would build differently with 3 months instead of 8 hours.
- Where your prototype would break at scale (1000s of accounts, multi-entity consolidation).
- How you used AI tools — where they helped, where they led you wrong.
- One thing about this problem you think we are underestimating.

5. What we are evaluating

Signal	What good looks like
Problem decomposition	Identifies that ‘generate financials’ is 6+ distinct sub-problems with different reliability requirements.
Agentic judgment	Knows when NOT to use an LLM. Treats agents as a tool, not a hammer.
Production thinking	Designs for traceability, idempotency, and human override from day one.
Domain humility	Asks clarifying questions about accounting semantics rather than guessing.
AI tool fluency	Uses AI to move faster without offloading judgment to it.

6. What will hurt you

- A polished demo on clean data with no failure handling.
- “I would just prompt a frontier model with the trial balance” as your architecture.
- Ignoring auditability. Finance software without traceability is unusable.
- Over-engineering. A 12-agent swarm for something a 200-line script could do is a red flag.
- Submitting with no clarifying questions. Knowing what to ask is part of the evaluation.

7. Submission

Send the architecture document as PDF or Markdown, the prototype as a Git repo (public or private invite) including a README with setup and run instructions, and the one-page reflection as plain text or PDF. If you have any output artifacts (generated statements, logs, traces), include them in the repo under */output*.

Good luck. We are looking forward to seeing how you think.